

Chapter 15: Query Processing

Outline

Basic Steps in Query Processing

Measures of Query Cost

Selection Operation

Sorting

Join Operation

Other Operations

Evaluation of Expressions

Basic Steps in Query Processing

Parsing and translation

translate the query into its internal form. This is then translated into **relational algebra**.

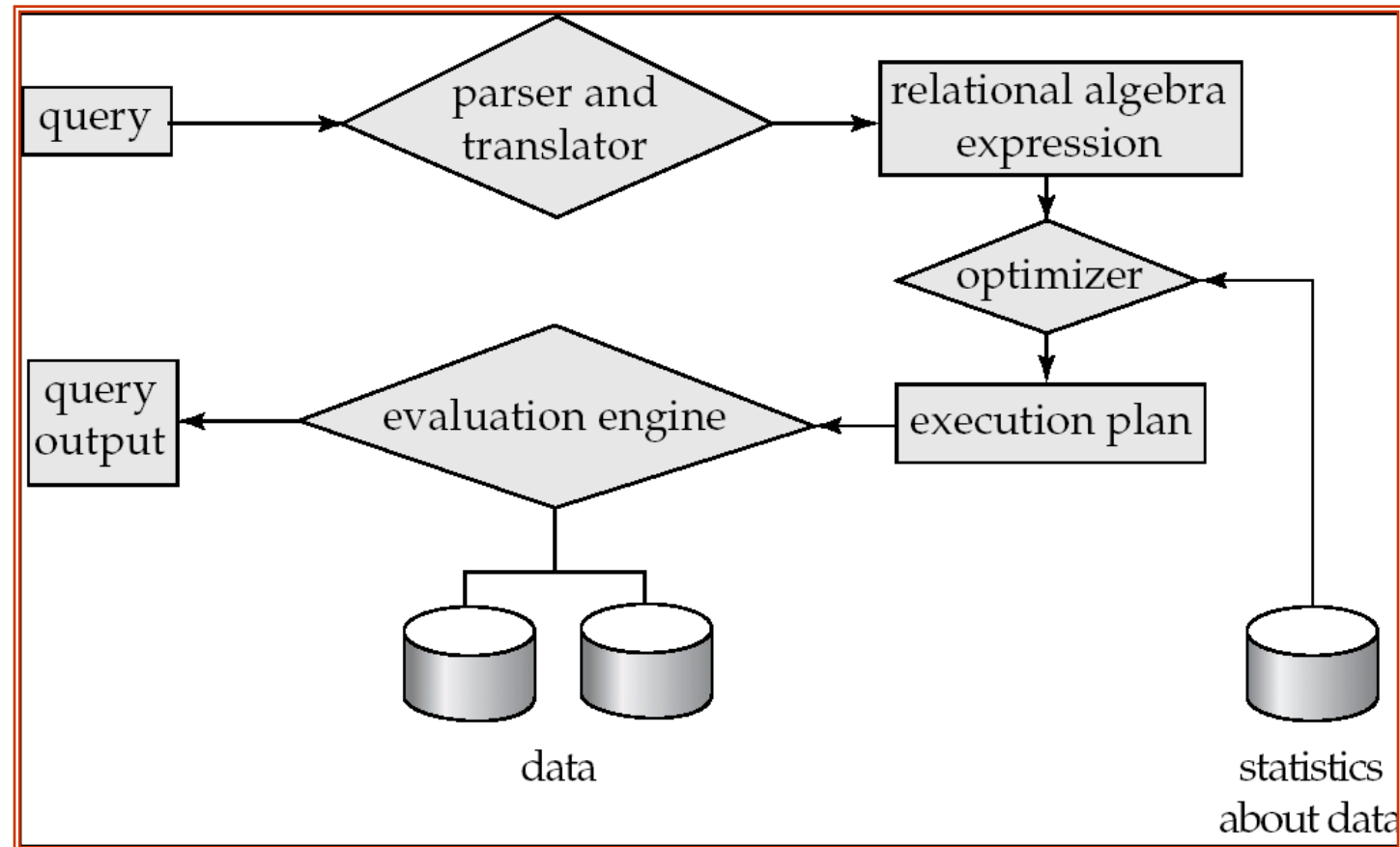
Parser checks syntax, verifies relations

Optimization

Amongst all equivalent **evaluation plans** choose the one with lowest cost.

Evaluation

The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.



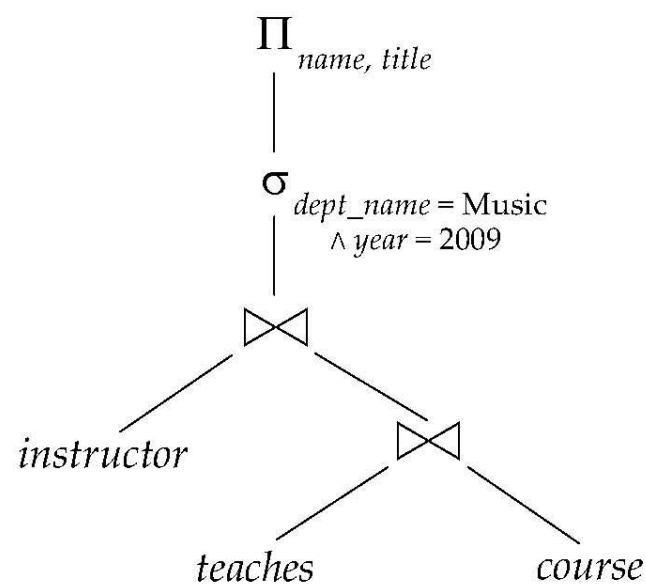
Basic Steps in Query Processing (Example)

select name, title

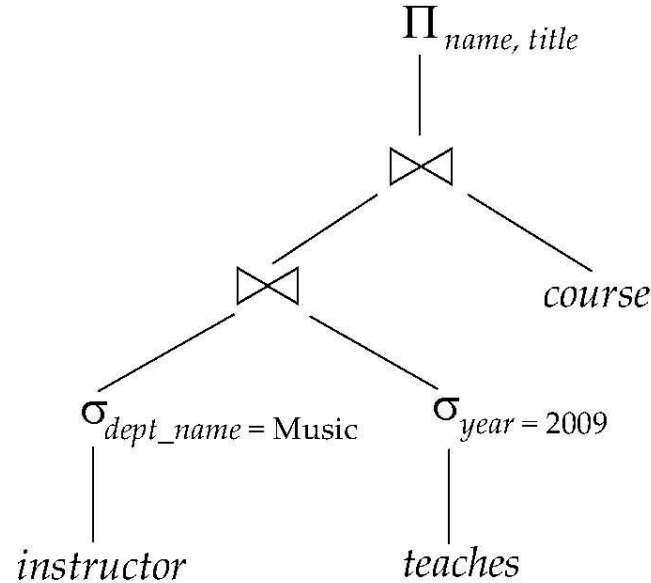
from instructor **natural join** (teaches **natural join** course)

where dept_name='Music' and year = 2009;

$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2009}$
 $(instructor \bowtie (teaches \bowtie (course))))$



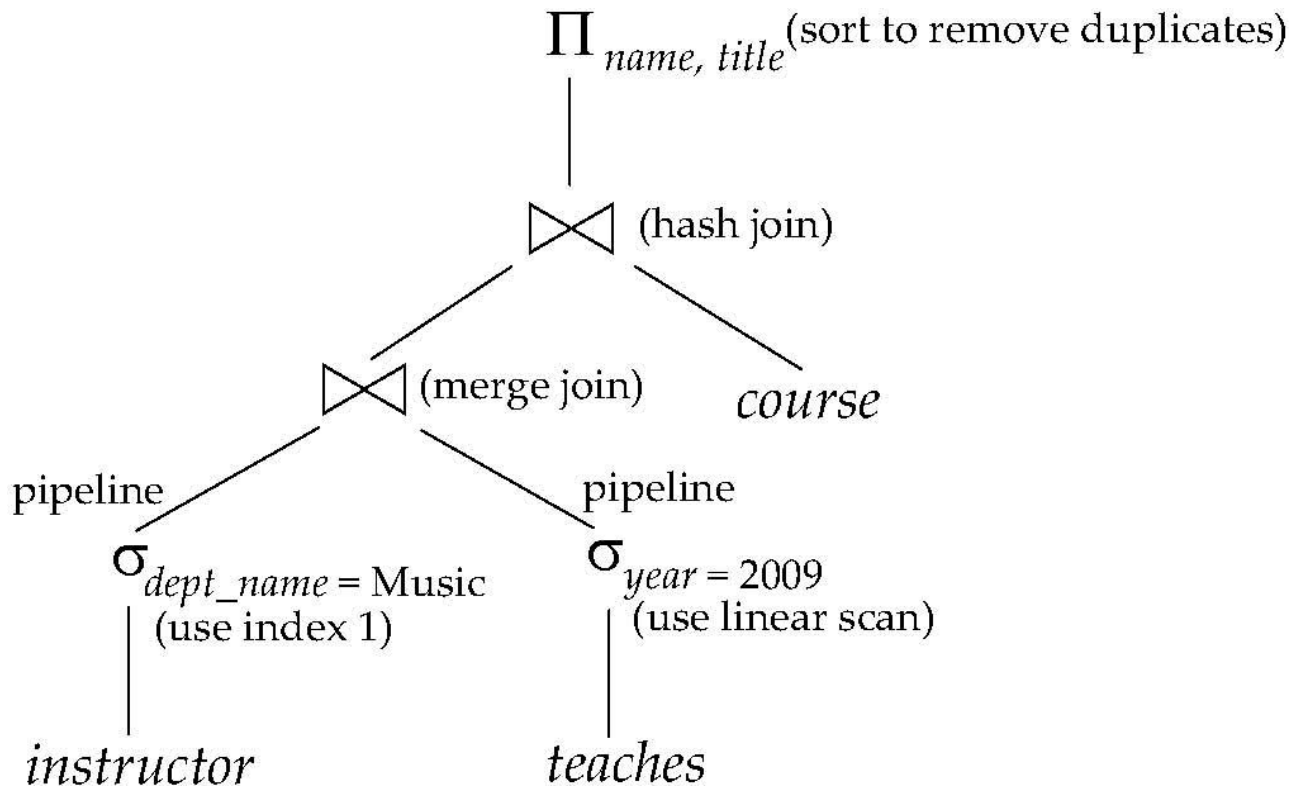
(a) Initial expression tree



(b) Tree after multiple transformations

Basic Steps in Query Processing

An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



Measures of Query Cost

Cost is generally measured as total elapsed time for answering query

Many factors contribute to time cost

- ▶ *disk accesses, CPU, or even network communication*

Typically disk access is the predominant cost, and is also relatively easy to estimate.

Measured by taking into account

Number of seeks * average-seek-cost

Number of blocks read * average-block-read-cost

Number of blocks written * average-block-write-cost

- ▶ Cost to write a block is greater than cost to read a block
 - data is read back after being written to ensure that the write was successful

Measures of Query Cost (Cont.)

For simplicity we just use the **number of block transfers** *from disk* and the **number of seeks** as the cost measures

t_T – time to transfer one block

t_S – time for one seek

Cost for b block transfers plus S seeks

$$b * t_T + S * t_S$$

t_S and t_T depend on where data is stored; with 4 KB blocks:

High end **magnetic disk**: $t_S = 4$ msec and $t_T = 0.1$ msec, $t_S / t_T \approx 40$

SSD: $t_S = 20$ -90 microsec and $t_T = 2$ -10 microsec for 4KB, $t_S / t_T \approx 10$

We **ignore CPU costs** for simplicity

Real systems do take CPU cost into account

We **ignore cost to writing output** to disk in our cost formulae

Measures of Query Cost (Cont.)

Several algorithms can reduce disk IO by using extra buffer space

Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution

- ▶ We often use **worst case estimates**, assuming only the minimum amount of memory needed for the operation is available

Required data may be buffer resident already, avoiding disk I/O

But hard to take into account for cost estimation

Selection Operation

File scan

Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.

$$\text{worst cost} = b_r * t_T + t_S$$

b_r denotes number of blocks containing records from relation r

If selection is on a **key attribute**, can stop on finding record

$$\text{average cost} = (b_r/2) t_T + t_S$$

Linear search can be applied regardless of

- ▶ selection condition or
- ▶ ordering of records in the file, or
- ▶ availability of indices

Note: binary search generally does not make sense since data is not stored consecutively

except when there is an index available,

and binary search requires more seeks than index search

Selections Using Indices

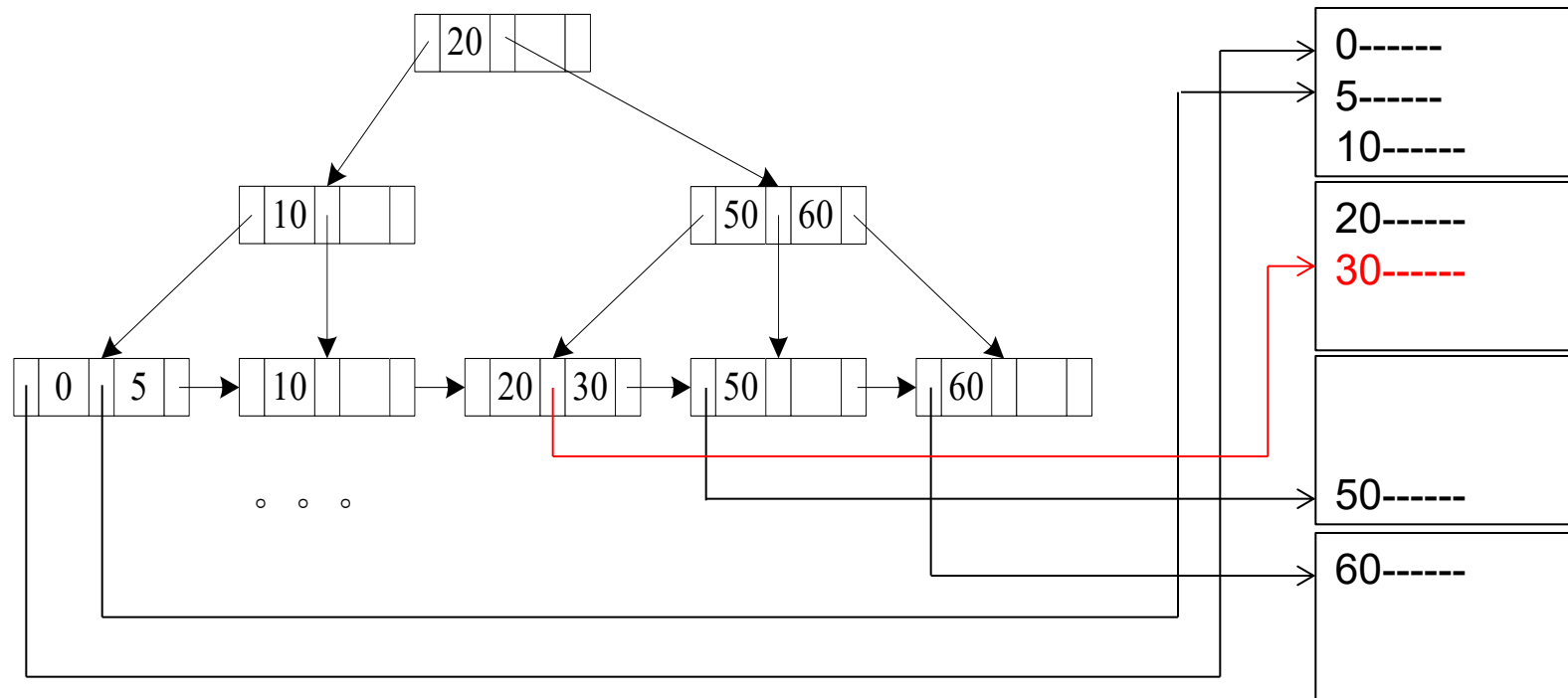
Index scan – search algorithms that use an index

selection condition must be on search-key of index.

A2 (primary B+-tree index / clustering B+-tree index, equality on key).

Retrieve a single record that satisfies the corresponding equality condition

$$\text{Cost} = (h_i + 1) * (t_T + t_S)$$



Selections Using Indices

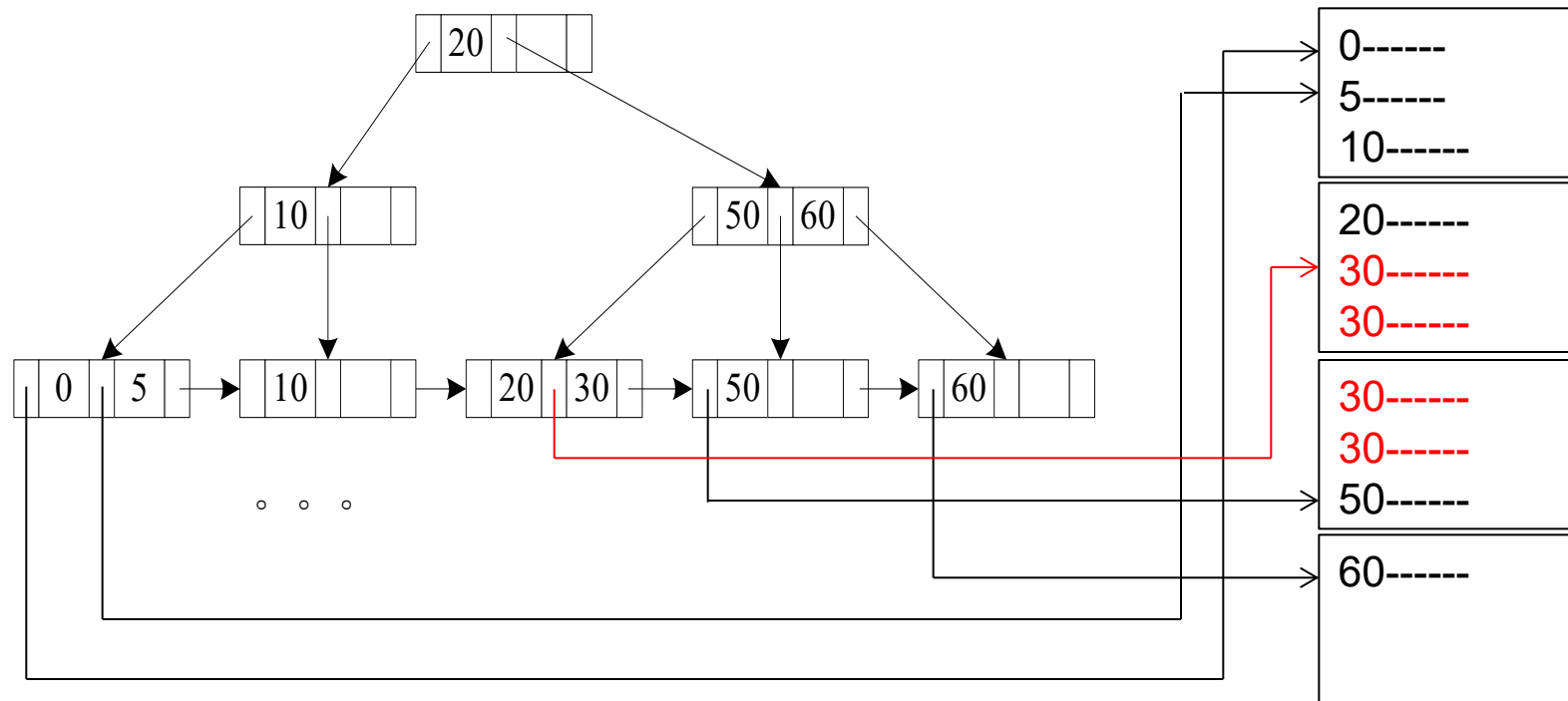
A3 (primary B+-tree index/ clustering B+-tree index, equality on nonkey)

Retrieve multiple records.

Records will be on consecutive blocks

- ▶ Let b = number of blocks containing matching records

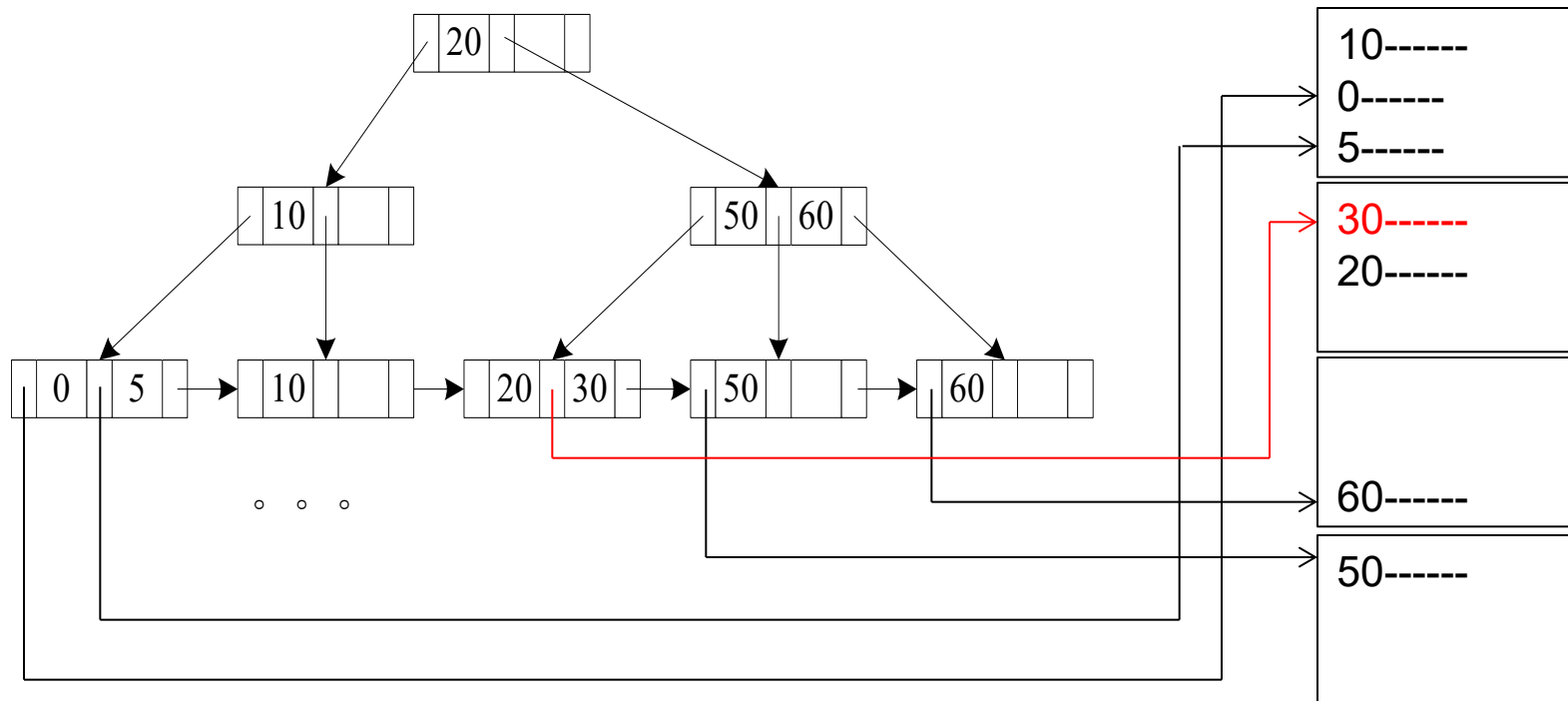
$$\text{Cost} = h_i * (t_T + t_S) + t_S + t_T * b$$



Selections Using Indices

A4 (secondary B+-tree index , equality on key) .

$$\text{Cost} = (h_i + 1) * (t_T + t_S)$$



Selections Using Indices

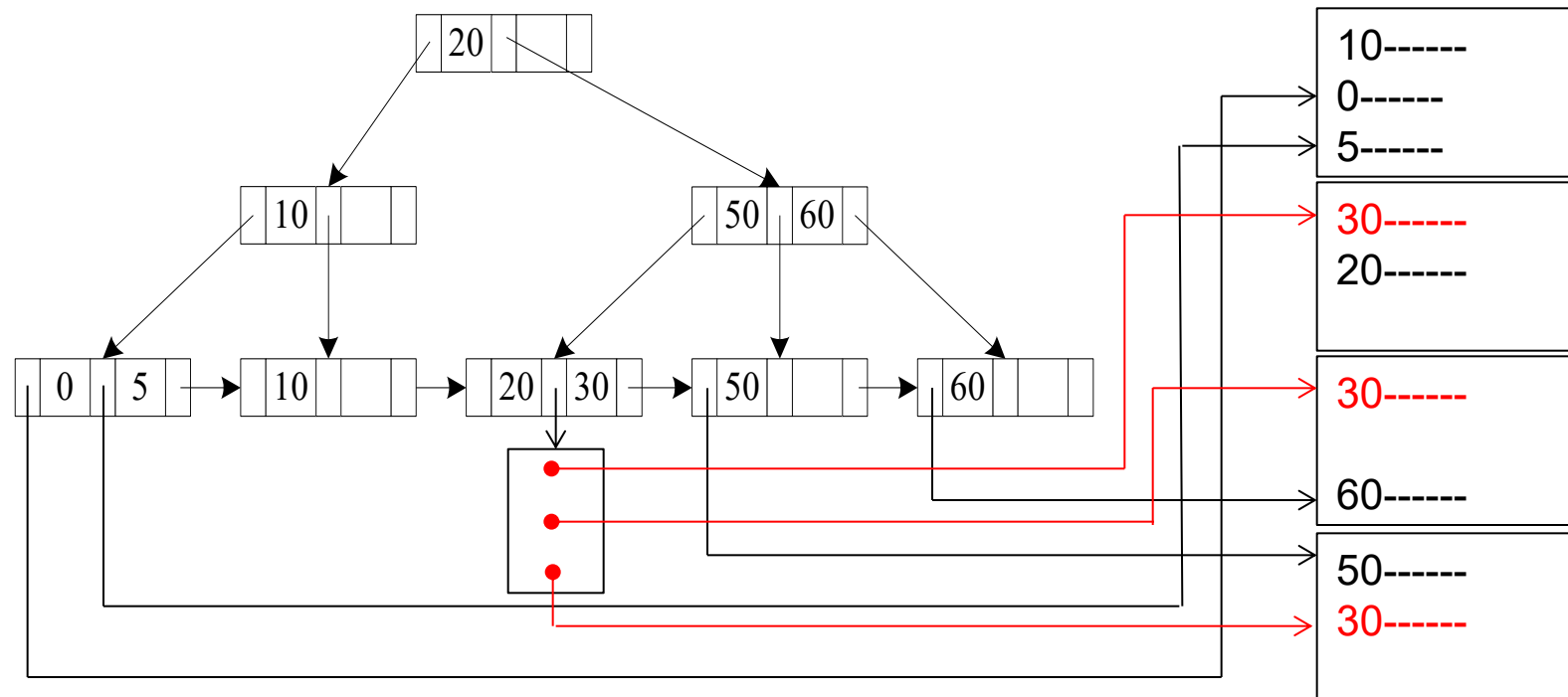
A4' (secondary B+-index on nonkey, equality).

Each of n matching records may be on a different block

n pointers may be stored in m blocks

$$\text{Cost} = (h_i + m + n) * (t_T + t_S)$$

— Can be very expensive!

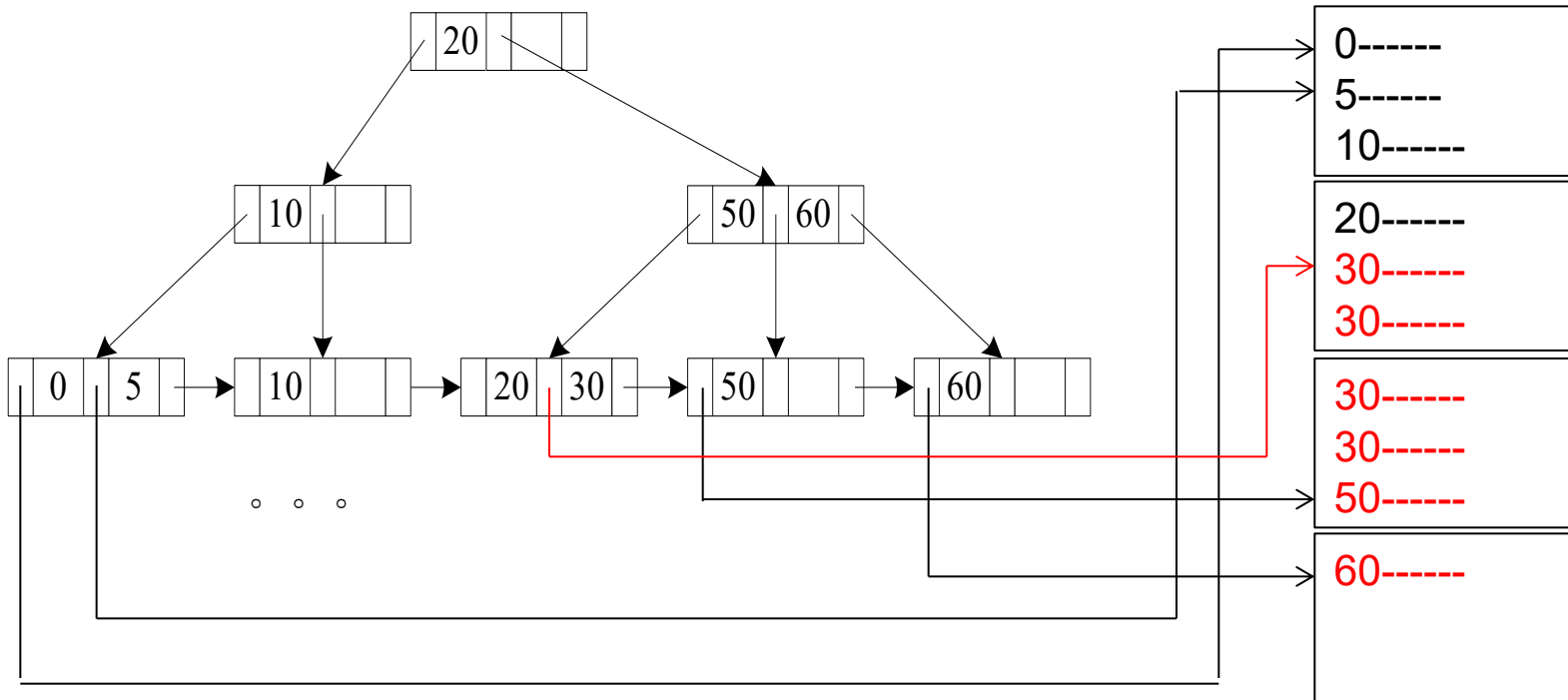


Selections Involving Comparisons

Can implement selections of the form $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$ by using
a linear file scan or binary search,
or by using indices in the following ways:

A5 (primary B+-index / clustering B+-index index, comparison). (Relation is sorted on A)

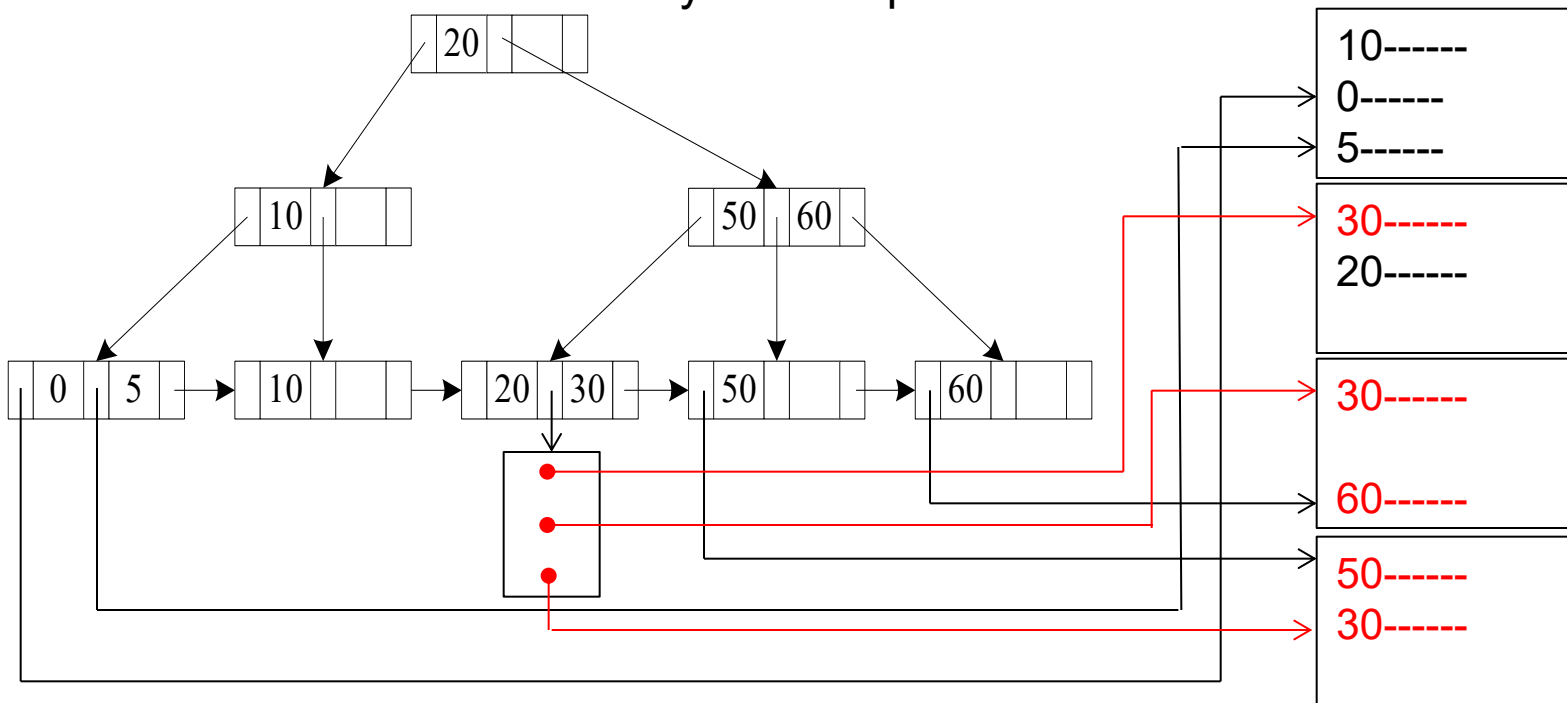
- ▶ For $\sigma_{A \geq V}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there.
- ▶ Cost is identical to the case of A3 $\rightarrow$ **Cost** = $h_i * (t_T + t_S) + t_S + t_T * b$
- ▶ For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$; do not use index



Selections Involving Comparisons

A6 (secondary B+-tree index, comparison).

- ▶ For $\sigma_{A \geq v}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
- ▶ For $\sigma_{A \leq v}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$
- ▶ In either case, retrieve records that are pointed to
 - requires an I/O for each record
 - Linear file scan may be cheaper



Implementation of Complex Selections

Conjunction: $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$

A7 (conjunctive selection using one index).

Select a combination of θ_i and algorithms A1 through A6 that results in the least cost for $\sigma_{\theta_i}(r)$.

Test other conditions on tuple after fetching it into memory buffer.

A8 (conjunctive selection using composite index).

Use appropriate composite (multiple-key) index if available.

A9 (conjunctive selection by intersection of identifiers).

Requires indices with record pointers.

Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers.

Then fetch records from file

If some conditions do not have appropriate indices, apply test in memory.

Algorithms for Complex Selections

Disjunction: $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$.

A10 (disjunctive selection by union of identifiers).

Applicable if *all* conditions have available indices.

- ▶ Otherwise use linear scan.

Use corresponding index for each condition, and take union of all the obtained sets of record pointers.

Then fetch records from file

Negation: $\sigma_{\neg\theta}(r)$

Use linear scan on file

If very few records satisfy $\neg\theta$, and an index is applicable to θ

- ▶ Find satisfying records using index and fetch from file

Sorting

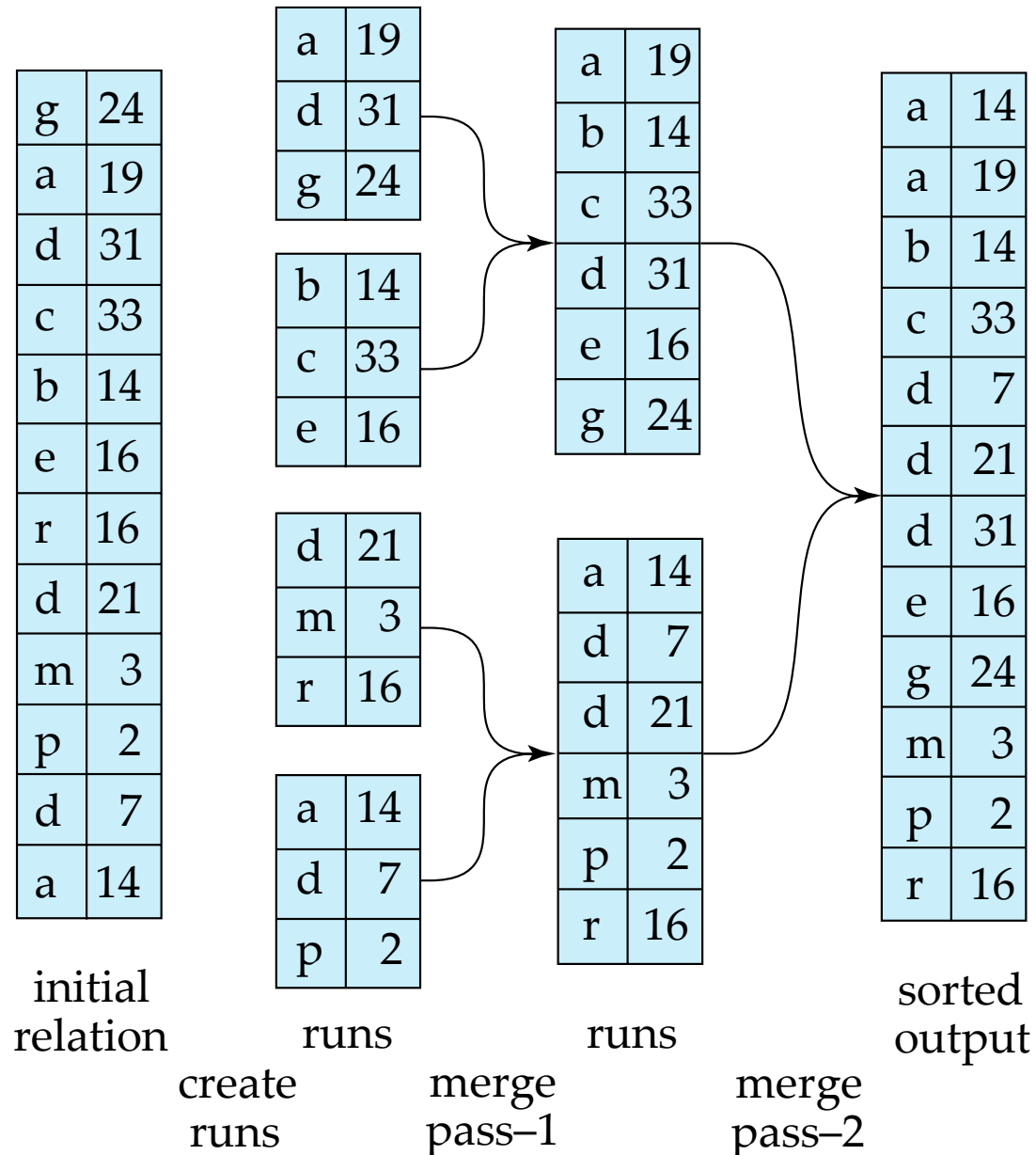
We may build an index on the relation, and then use the index to read the relation in sorted order. May lead to one disk block access for each tuple.

For relations that fit in memory, techniques like quicksort can be used.

For relations that don't fit in memory,

external sort-merge is a good choice.

Example: External Sorting Using Sort-Merge



External Sort-Merge

Let M denote memory size (in pages).

1. Create sorted runs(归并段).

Let i be 0 initially.

Repeatedly do the following till the end of the relation:

- (a) Read M blocks of relation into memory
- (b) Sort the in-memory blocks
- (c) Write sorted data to run R_i ; increment i .

Let the final value of i be N

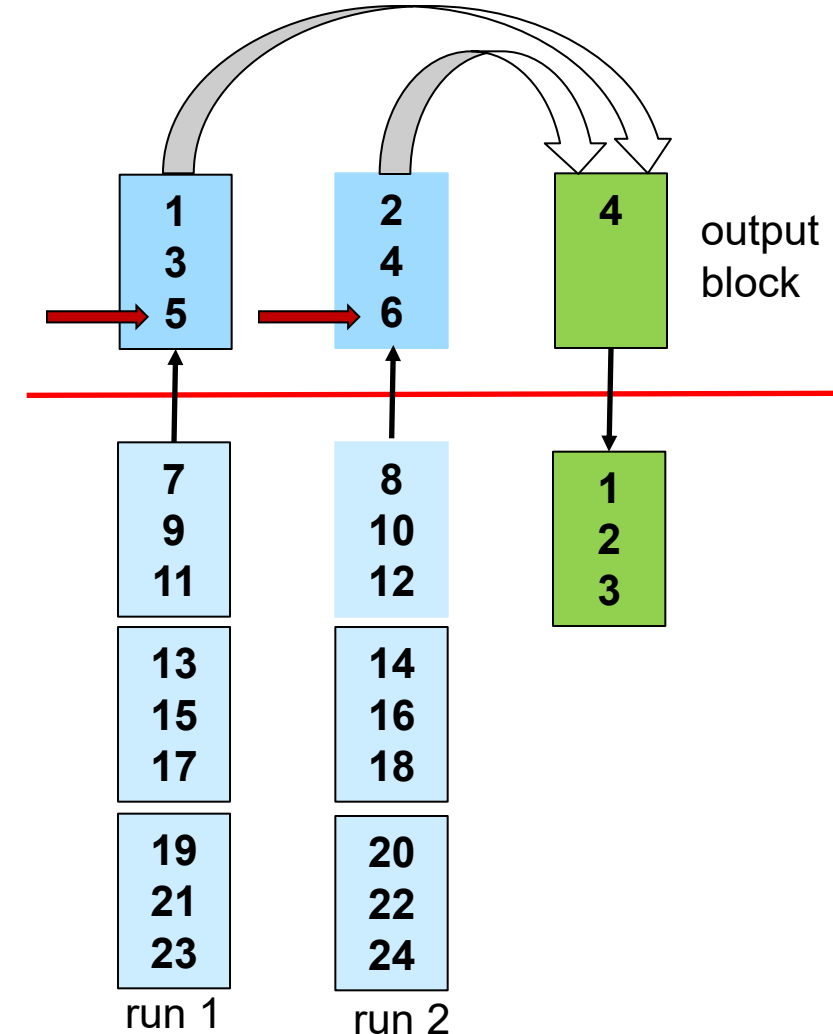
External Sort-Merge (Cont.)

2. Merge the runs (N-way merge).

IF $N < M$, single merge pass is required

(如果归并段少于可用内存页)

1. Use N blocks of memory to buffer input runs, and 1 block to buffer output. Read the first block of each run into its buffer page
2. repeat
 1. Select the first record (in sort order) among all buffer pages
 2. Write the record to the output buffer. If the output buffer is full write it to disk.
 3. Delete the record from its input buffer page.
If the buffer page becomes empty then read the next block (if any) of the run into the buffer.
3. until all input buffer pages are empty:



External Sort-Merge (Cont.)

If $N \geq M$, several merge *passes* are required.

In each **pass**, contiguous groups of $M - 1$ runs are merged.

A pass reduces the number of runs by a factor of $M - 1$, and creates runs longer by the same factor.

- ▶ E.g. If $M=11$, and there are 90 runs, one pass reduces the number of runs to 9, each 10 times the size of the initial runs

Repeated passes are performed till all runs have been merged into one.

External Merge Sort (Cont.)

Cost analysis: (*simple version*)

Total number of **runs** : $\lceil b_r / M \rceil$.

Total number of merge **passes** required: $\lceil \log_{M-1}(b_r / M) \rceil$.

Block transfers for initial run creation as well as in each pass is $2b_r$

- ▶ for final pass, we don't count write cost
 - we ignore final write cost for all operations since the output of an operation may be sent to the parent operation without being written to disk
- ▶ Thus total number of **block transfers** for external sorting:

$$\begin{aligned} & 2b_r \lceil \log_{M-1}(b_r / M) \rceil + b_r \\ &= b_r (2 \lceil \log_{M-1}(b_r / M) \rceil + 1) \end{aligned}$$

External Merge Sort (Cont.)

Cost of **seeks**: (*simple version*)

During run generation: one seek to read each run and one seek to write each run

- ▶ $2 \lceil b_r / M \rceil \rightarrow 2 * [\text{the number of runs}]$

During the merge phase

- ▶ Need $2 b_r$ seeks for each merge pass
 - except the final one which does not require a write
- ▶ There are $\lceil \log_{M-1} (b_r / M) \rceil$ merger passes
- ▶ Total number of seeks:
 $2 \lceil b_r / M \rceil + b_r (2 \lceil \log_{M-1} (b_r / M) \rceil - 1)$

External Merge Sort (Cont.)

Cost analysis: (advanced version)

1 block per run leads to too many seeks during merge

▶ **Instead use b_b buffer blocks per run**

→ read/write b_b blocks at a time

▶ Can merge $\lfloor M/b_b \rfloor - 1$ runs in one pass

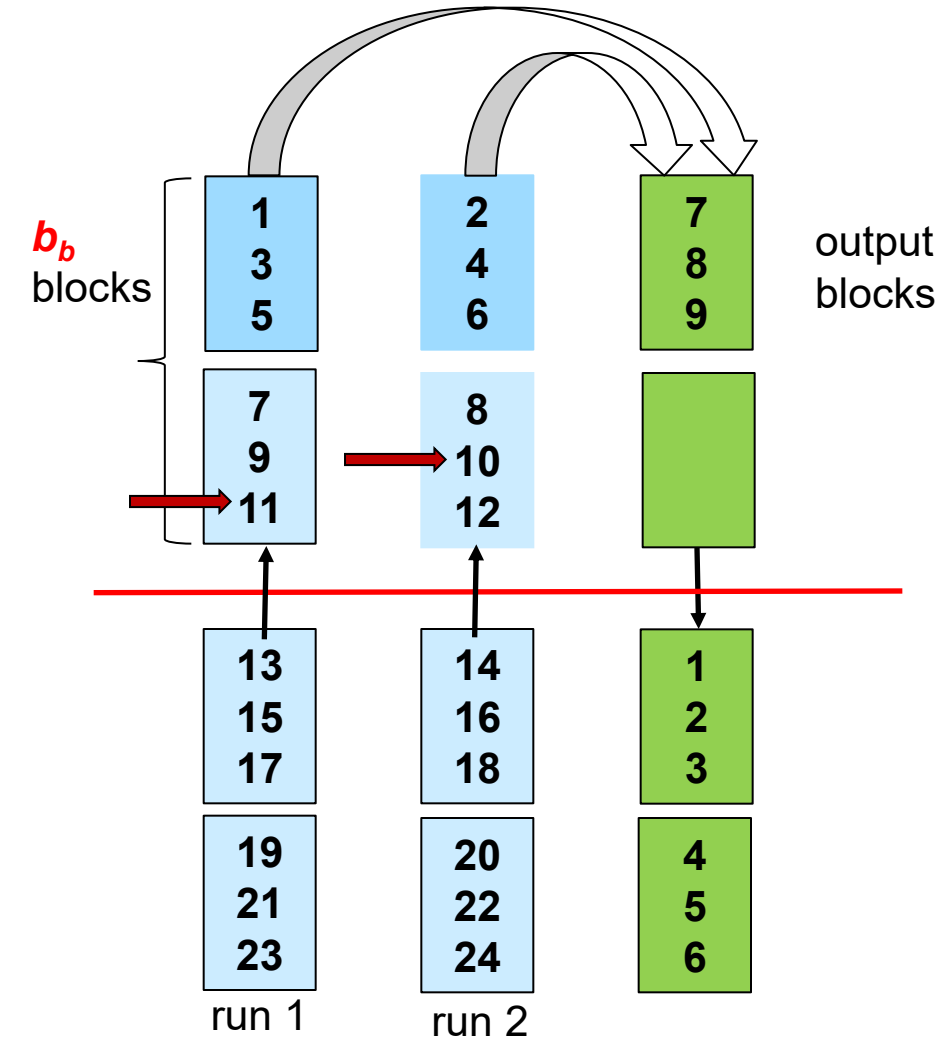
Total number of merge passes required:

$$\lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r / M) \rceil.$$

Block transfers for initial run creation as well as in each pass is $2b_r$

- ▶ for final pass, we don't count write cost
 - we ignore final write cost for all operations since the output of an operation may be sent to the parent operation without being written to disk
- ▶ Thus total number of block transfers for external sorting:

$$b_r (2 \lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r / M) \rceil + 1)$$



External Merge Sort (Cont.)

Cost of **seeks**: (advanced version)

During run generation: one seek to read each run and one seek to write each run

- ▶ $2 \lceil br / M \rceil \rightarrow 2 \times [\text{the number of runs}]$

During the merge phase

- ▶ Need $2 \lceil br / b_b \rceil$ seeks for each merge pass
 - except the final one which does not require a write
- ▶ There are $\lceil \log_{\lfloor M/b_b \rfloor - 1} (br / M) \rceil$ merger passes
- ▶ Total number of seeks:
 $2 \lceil br / M \rceil + \lceil br / b_b \rceil (2 \lceil \log_{\lfloor M/b_b \rfloor - 1} (br / M) \rceil - 1)$

Join Operation

Several different algorithms to implement joins

Nested-loop join

Block nested-loop join

Indexed nested-loop join

Merge-join

Hash-join

Choice based on cost estimate

Examples use the following information

Number of **records** of *student*: 5,000 , *takes*: 10,000

Number of **blocks** of *student*: 100 , *takes*: 400

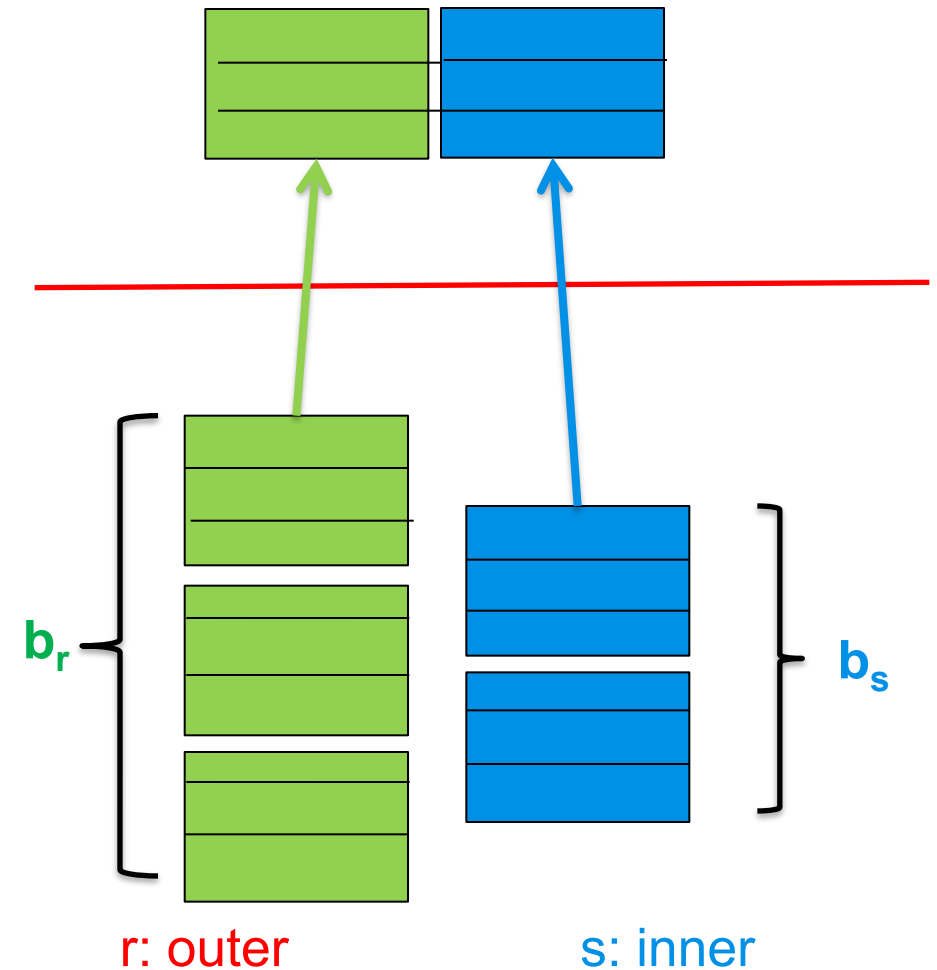
Nested-Loop Join

To compute the theta join $r \bowtie_{\theta} s$
for each tuple t_r **in** r **do begin**
 for each tuple t_s **in** s **do begin**
 test pair (t_r, t_s) to see if they satisfy the
 join condition θ
 if they do, add $t_r \cdot t_s$ to the result.
 end
end

r is called the **outer relation** and s the **inner relation** of the join.

Requires no indices and can be used with any kind of join condition.

Expensive since it examines every pair of tuples in the two relations.



Nested-Loop Join (Cont.)

In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is

$$\begin{array}{ll} n_r * b_s + b_r & \text{block transfers, plus} \\ n_r + b_r & \text{seeks} \end{array}$$

If the smaller relation fits entirely in memory, use that as the inner relation.

Reduces cost to $b_r + b_s$ block transfers and 2 seeks

Assuming worst case memory availability cost estimate is

with *student* as outer relation:

- ▶ $5000 * 400 + 100 = 2,000,100$ block transfers,
- ▶ $5000 + 100 = 5100$ seeks

with *takes* as the outer relation

- ▶ $10000 * 100 + 400 = 1,000,400$ block transfers and 10,400 seeks

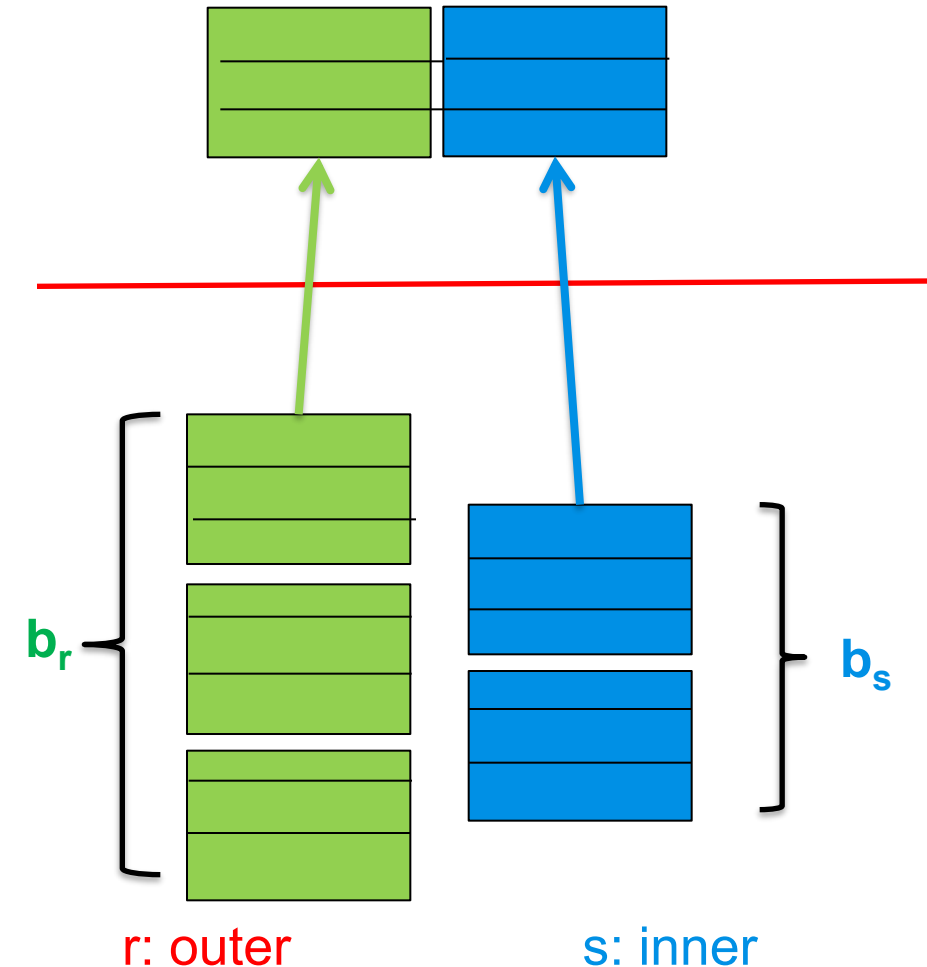
If smaller relation (*student*) fits entirely in memory, the cost estimate will be 500 block transfers.

Block nested-loops algorithm (next slide) is preferable.

Block Nested-Loop Join

Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

```
for each block  $B_r$  of  $r$  do begin
  for each block  $B_s$  of  $s$  do begin
    for each tuple  $t_r$  in  $B_r$  do begin
      for each tuple  $t_s$  in  $B_s$  do begin
        Check if  $(t_r, t_s)$  satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result.
      end
    end
  end
end
```



Block Nested-Loop Join (Cont.)

Worst case estimate: $b_r * b_s + b_r$ block transfers + $2 * b_r$ seeks

Each block in the inner relation s is read once for each *block* in the outer relation

Best case: $b_r + b_s$ block transfers + 2 seeks.

Block Nested-Loop Join (Cont.)

Improvements to block nested loop algorithms:

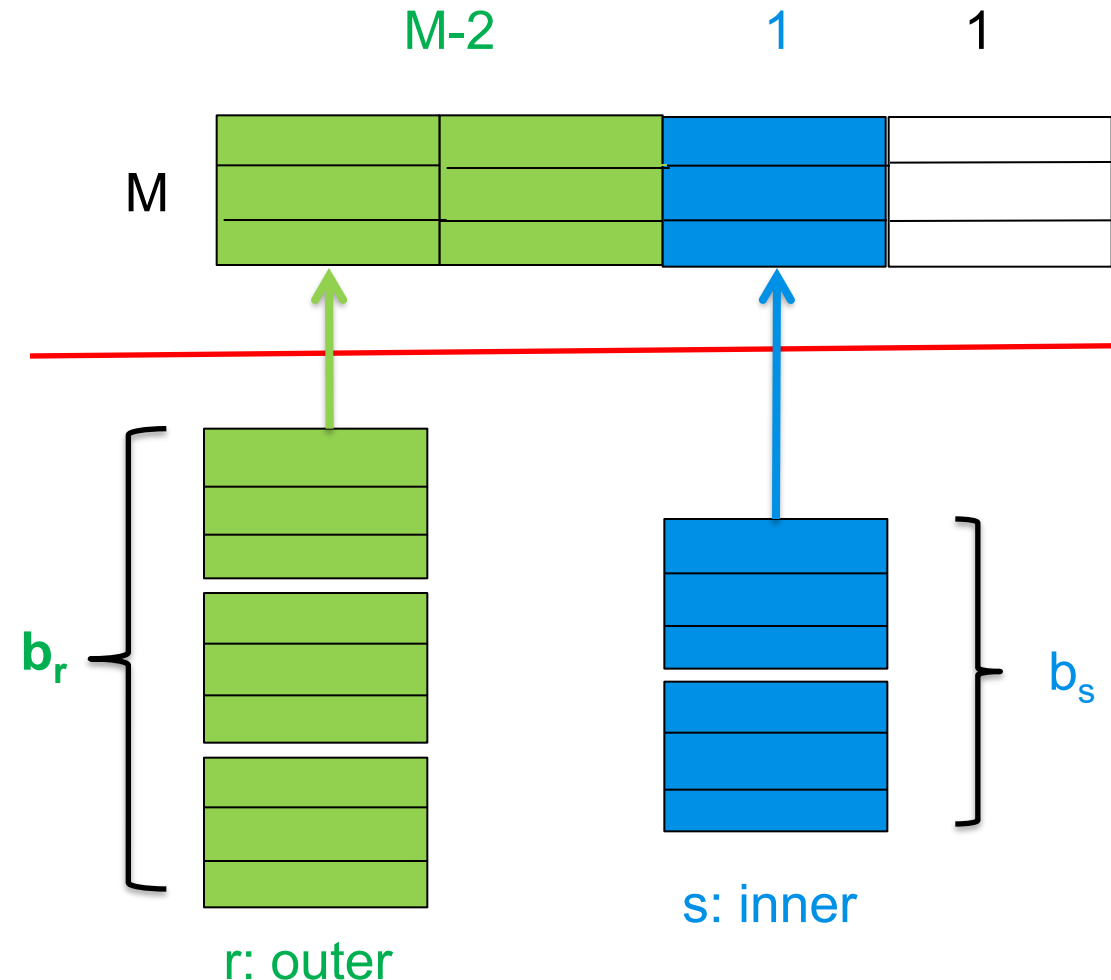
Use $M-2$ disk blocks as blocking unit for outer relations, where M = memory size in blocks; use remaining two blocks to buffer inner relation and output

- ▶ $\text{Cost} = \lceil b_r / (M-2) \rceil * b_s + b_r$ block transfers
+ $2 \lceil b_r / (M-2) \rceil$ seeks
- ▶ If r fits in memory ($b_r \leq M-2$),

$$\text{Cost} = b_s + b_r \text{ block transfers} + 2 \text{ seeks}$$

If equi-join attribute forms a key on the inner relation, stop inner loop on first match

Scan inner loop forward and backward alternately, to make use of the blocks remaining in buffer (with LRU replacement)



Indexed Nested-Loop Join

Index lookups can replace file scans if

join is an equi-join or natural join and

an index is available on the inner relation's join attribute

- ▶ Can construct an index just to compute a join.

For each tuple t_r in the outer relation r , use the index to look up tuples in s that satisfy the join condition with tuple t_r .

Worst case: buffer has space for only one page of r , and, for each tuple in r , we perform an index lookup on s .

Cost of the join: $b_r (t_T + t_S) + n_r * c$

Where c is the cost of traversing index and fetching all matching s tuples for one tuple of r

c can be estimated as cost of a single selection on s using the join condition.

If indices are available on join attributes of both r and s , use the relation with fewer tuples as the outer relation.

Merge-Join

1. Sort both relations on their **join attribute** (if not already sorted on the join attributes).
2. Merge the sorted relations to join them
 1. Join step is similar to the merge stage of the sort-merge algorithm.
 2. Main difference is handling of **duplicate** values in join attribute — every pair with same value on join attribute must be matched
3. Detailed algorithm in book

pr →

<i>a1</i>	<i>a2</i>
a	3
b	1
d	8
d	13
f	7
m	5
q	6

r

ps →

<i>a1</i>	<i>a3</i>
a	A
b	G
c	L
d	N
m	B

s

Merge-Join (Cont.)

Can be used only for equi-joins and natural joins

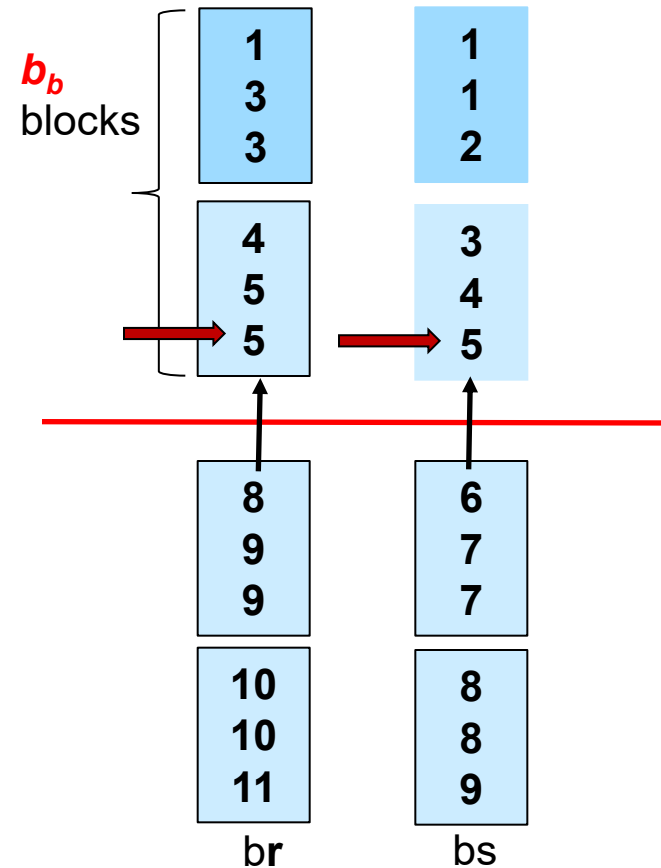
Each block needs to be read only once (assuming all tuples for any given value of the join attributes fit in memory)

Thus the cost of merge join is:

$b_r + b_s$ block transfers +

$\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil$ seeks

+ the cost of sorting if relations are unsorted.



Merge-Join (Cont.)

If the buffer memory size is M pages, in order to minimize the cost of merge join, how to assign M blocks to r and s respectively?

The estimated cost is

$b_r + b_s$ block transfers +

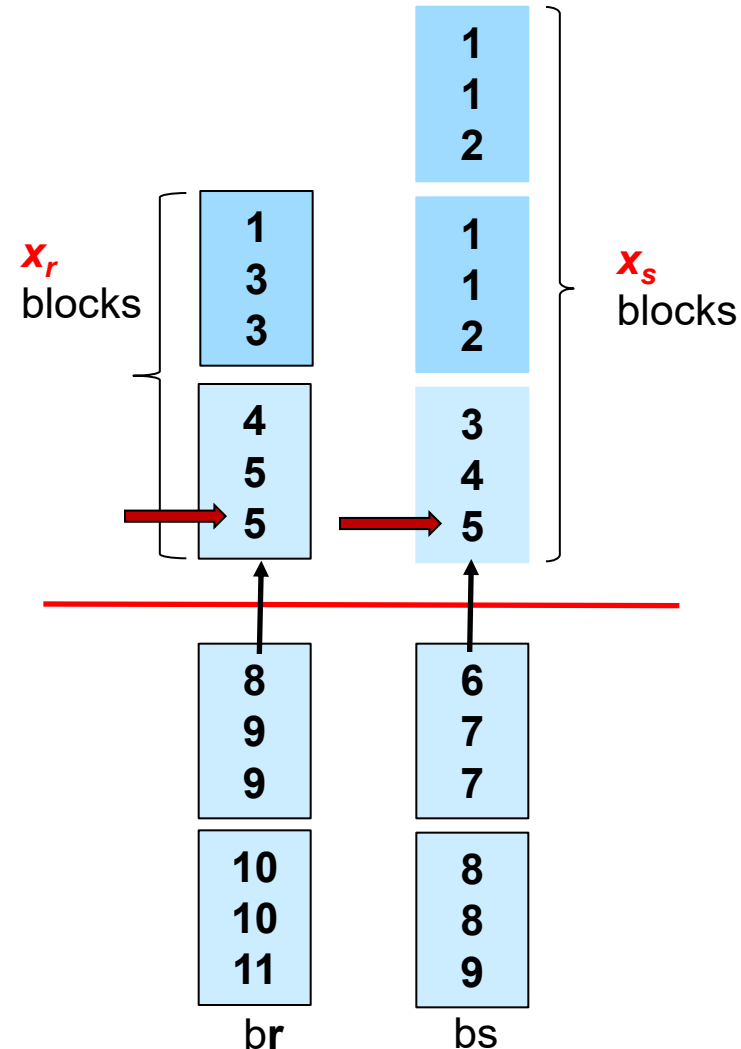
$\lceil b_r/x_r \rceil + \lceil b_s/x_s \rceil$ seeks

$(x_r + x_s = M)$

Answer:

$$x_r = \sqrt{b_r * M / (\sqrt{b_r} + \sqrt{b_s})}$$

$$x_s = \sqrt{b_s * M / (\sqrt{b_r} + \sqrt{b_s})}$$



Merge-Join (Cont.)

hybrid merge-join: If one relation is sorted, and the other has a secondary B⁺-tree index on the join attribute

Merge the sorted relation with the leaf entries of the B⁺-tree .

Sort the result on the addresses of the unsorted relation's tuples

Scan the unsorted relation in physical address order and merge with previous result, to replace addresses by the actual tuples

- ▶ Sequential scan more efficient than random lookup

Hash-Join

Applicable for **equi-joins** and **natural joins**.

A **hash function** h is used to partition tuples of both relations with common **JoinAttrs**

h maps *JoinAttrs* values to $\{0, 1, \dots, n\}$,

$r_0, r_1, \dots, r_n$ denote partitions of r tuples

- ▶ Each tuple $t_r \in r$ is put in partition r_i where $i = h(t_r[\text{JoinAttrs}])$.

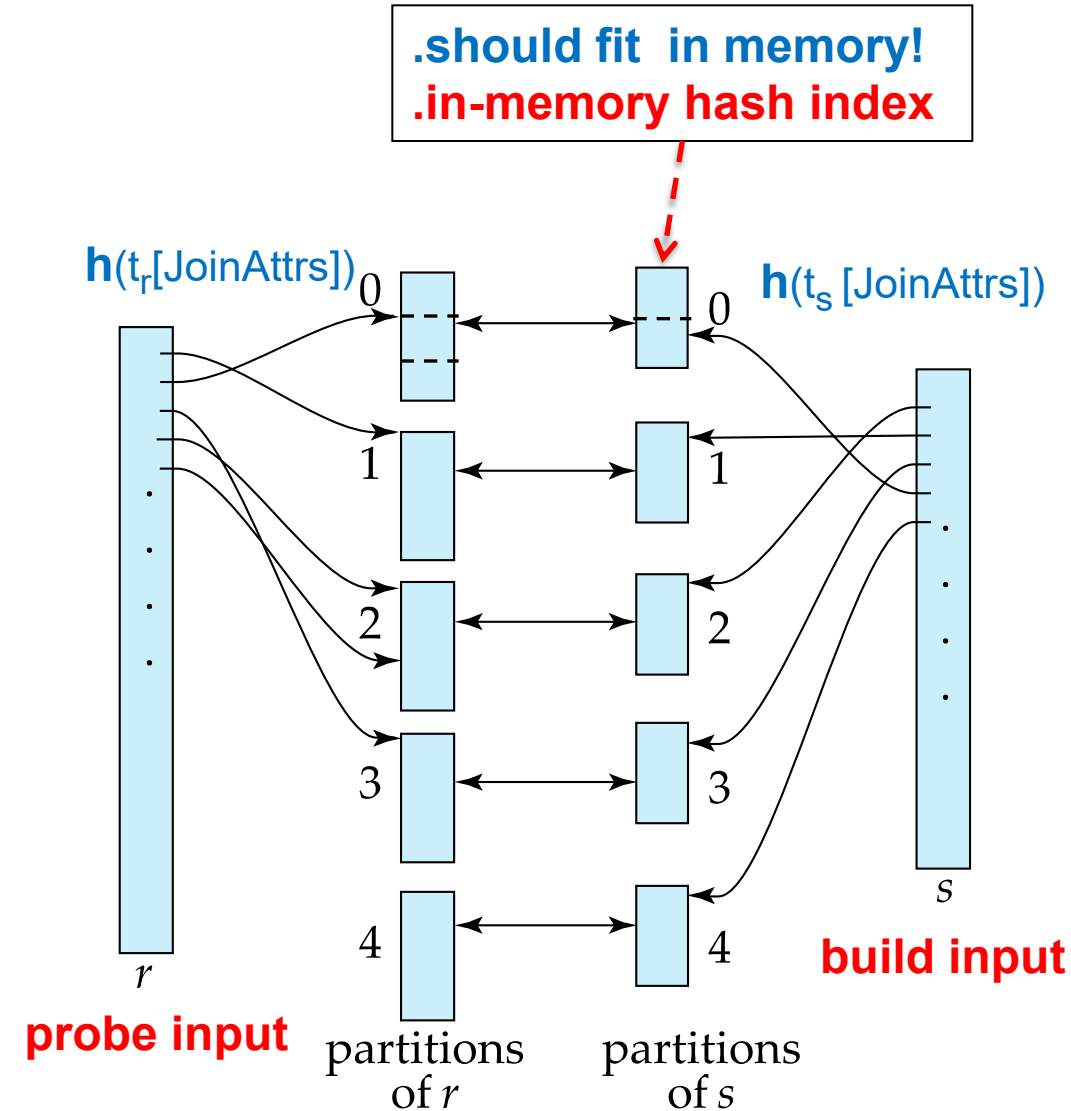
$s_0, s_1, \dots, s_n$ denotes partitions of s tuples

- ▶ Each tuple $t_s \in s$ is put in partition s_i , where $i = h(t_s[\text{JoinAttrs}])$.

*Note: the value n and the hash function h is chosen such that **each s_i should fit in memory**.*

$$n \geq \lceil b_s / M \rceil$$

Note: In book, r_i is denoted as H_{ri} , s_i is denoted



Hash-Join (Cont.)

r tuples in r_i need only to be compared with s tuples in s_i

Need not be compared with s tuples in any other partition, since:

an r tuple and an s tuple that satisfy the join condition will have the same value for the join attributes.

If that value is hashed to some value i , the r tuple has to be in r_i and the s tuple in s_i .

Hash-Join Algorithm

The hash-join of r and s is computed as follows.

1. **Partition s** using hashing function h . When partitioning a relation, one block of memory is reserved as the output buffer for each partition.
2. **Partition r** similarly.
3. For each i :
 - (a) Load s_i into memory and build **an in-memory hash index** on it using the join attribute. This hash index uses a **different hash function** than the earlier one h .
 - (b) Read the tuples in r_i from the disk one by one. For each tuple t_r locate each matching tuple t_s in s_i using the in-memory hash index. Output the concatenation of their attributes.

Relation s is called the **build input**

Relation r is called the **probe input**

Hash-Join algorithm (Cont.)

The value n and the hash function h is chosen such that each build input relation partitions s_i should fit in memory.

Typically n is chosen as $\lceil b_s/M \rceil * f$ where f is a “fudge factor(修正因子)”, typically around 1.2

The probe input relation partitions r_i need not fit in memory

Recursive partitioning

Recursive partitioning required if number of partitions n is greater than number of pages M of memory.

Instead of partitioning n ways, use $M - 1$ partitions for s

Further partition the $M - 1$ partitions using a different hash function

Use same partitioning method on r

A relation does not need recursive partitioning if $M > n_h + 1$, or equivalently $M > (b_s/M) + 1$, which simplifies (approximately) to $M > \sqrt{b_s}$.

For example, consider a memory size of 12M bytes, divided into 4K bytes blocks; it would contain a total of 3K (3072) blocks. We can use a memory of this size to partition relations of size up to $3K * 3K$ blocks, which is 36G ($3K * 3K * 4K$) bytes.

Similarly, a relation of size 1 gigabyte requires just over $\sqrt{256K}$ blocks, or 2 megabytes, to avoid recursive partitioning.

Cost of Hash-Join

If recursive partitioning is not required: cost of hash join is

$$3(b_r + b_s) + 4 * n_h \text{ block transfers} + \\ 2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) + 2 * n_h \text{ seeks}$$

Block transfers : The partitioning of the two relations r and s calls for a complete reading of both relations, and a subsequent writing back of them. This operation requires $2(b_r + b_s)$ block transfers. The build and probe phases read each of the partitions once, calling for further $b_r + b_s$ block transfers. The number of blocks occupied by partitions could be slightly more than $b_r + b_s$, as a result of partially filled blocks. Accessing such partially filled blocks can add an overhead of at most $2n_h$ for each of the relations, since each of the n_h partitions could have a partially filled block that has to be written and read back. Thus, a hash join is estimated to require: $3(b_r + b_s) + 4n_h$ block transfers .

Seeks: Assuming b_b blocks are allocated for the input buffer and each output buffer, partitioning requires a total of $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil)$ seeks. The build and probe phases require only one seek for each of the n_h partitions of each relation, since each partition can be read sequentially. The hash join thus requires $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) + 2n_h$ seeks.

Cost of Hash-Join

If recursive partitioning required:

number of passes required for partitioning build relation s to less than M blocks per partition is $\lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil$

best to choose the smaller relation as the build relation.

Total cost estimate is:

$$2(b_r + b_s) \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil + b_r + b_s \text{ block transfers} + \\ 2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil) \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil \text{ seeks}$$

If the entire build input can be kept in main memory no partitioning is required

Cost estimate goes down to $b_r + b_s$.

Other Operations

Duplicate elimination can be implemented via hashing or sorting.

On sorting duplicates will come adjacent to each other, and all but one set of duplicates can be deleted.

Optimization: duplicates can be deleted during run generation as well as at intermediate merge steps in external sort-merge.

Hashing is similar – duplicates will come into the same bucket.

Projection:

perform projection on each tuple
followed by duplicate elimination.

Other Operations : Aggregation

Aggregation can be implemented in a manner similar to duplicate elimination.

Sorting or hashing can be used to bring tuples in the same group together, and then the aggregate functions can be applied on each group.

Optimization: combine tuples in the same group during run generation and intermediate merges, by computing partial aggregate values

- ▶ For count, min, max, sum: keep aggregate values on tuples found so far in the group.
 - When combining partial aggregate for count, add up the aggregates
- ▶ For avg, keep sum and count, and divide sum by count at the end

Other Operations : Set Operations

Set operations ($\cup$, $\cap$ and $-$): can either use variant of merge-join after sorting, or variant of hash-join.

E.g., Set operations using hashing:

1. Partition both relations using the same hash function
2. Process each partition i as follows.
 1. Using a different hashing function, build an in-memory hash index on r_i .
 2. Process s_i as follows
 $r \cup s$:
 1. Add tuples in s_i to the hash index if they are not already in it.
 2. At end of s_i add the tuples in the hash index to the result.

Other Operations : Set Operations

E.g., Set operations using hashing:

1. as before partition r and s ,
2. as before, process each partition i as follows
 1. build a hash index on r_i
 2. Process s_i as follows

$r \cap s$:

1. output tuples in s_i to the result if they are already there in the hash index

$r - s$:

1. for each tuple in s_i , if it is there in the hash index, delete it from the index.
2. At end of s_i add remaining tuples in the hash index to the result.

Other Operations : Outer Join

Outer join can be computed either as

A join followed by addition of null-padded non-participating tuples.

by modifying the join algorithms.

Modifying merge join to compute $r \bowtie s$

In $r \bowtie s$, non participating tuples are those in $r - \Pi_R(r \bowtie s)$

Modify merge-join to compute $r \bowtie s$:

- ▶ During merging, for every tuple t_r from r that do not match any tuple in s , output t_r padded with nulls.

Right outer-join and full outer-join can be computed similarly.

Other Operations : Outer Join

Modifying hash join to compute $r \bowtie s$

If r is probe relation, output non-matching r tuples padded with nulls

If r is build relation, when probing keep track of which r tuples matched s tuples. At end of s_i output non-matched r tuples padded with nulls

Evaluation of Expressions

So far: we have seen algorithms for individual operations

Alternatives for evaluating an entire expression tree

Materialization(物化): generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.

Pipelining(流水线): pass on tuples to parent operations even as an operation is being executed

We study above alternatives in more detail

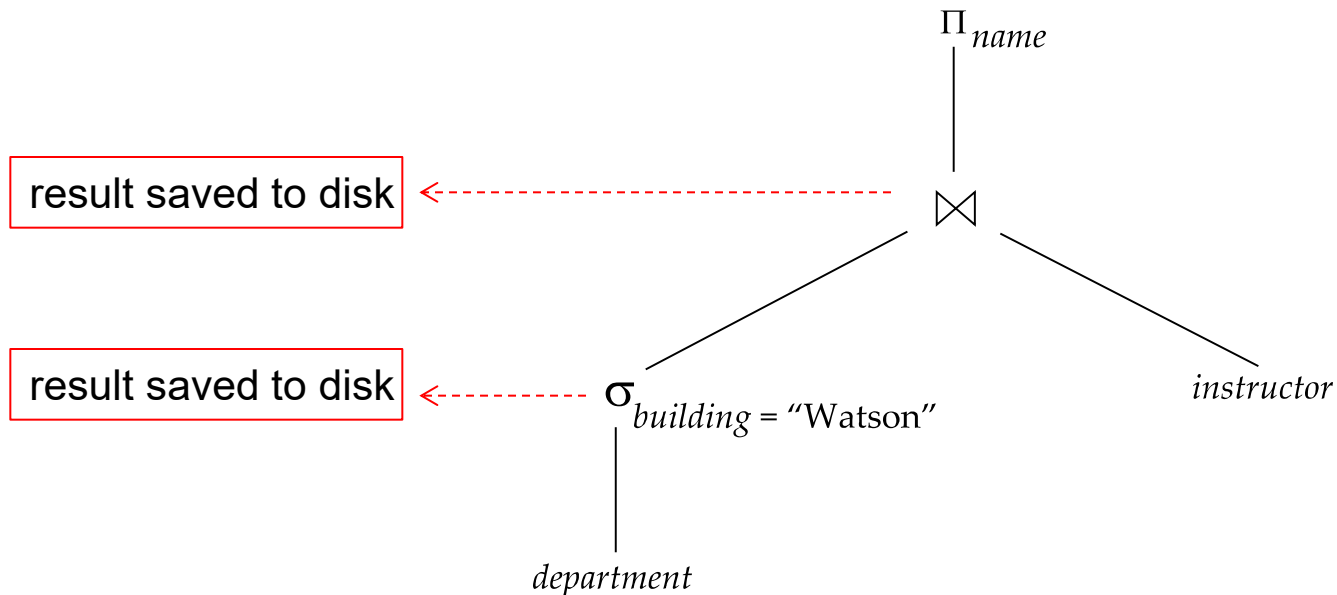
Materialization

Materialized evaluation: evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.

E.g., in figure below, compute and store

$$\sigma_{building="Watson"}(department)$$

then compute the store its join with *instructor*, and finally compute the projection on *name*.



Materialization (Cont.)

Materialized evaluation is always applicable

Cost of writing results to disk and reading them back can be quite high

Our cost formulas for operations ignore cost of writing results to disk, so

- ▶ Overall cost = Sum of costs of individual operations + cost of writing intermediate results to disk

Double buffering: use two output buffers for each operation, when one is full write it to disk while the other is getting filled

Allows overlap of disk writes with computation and reduces execution time

Pipelining

Pipelined evaluation : evaluate several operations simultaneously, passing the results of one operation on to the next.

E.g., in previous expression tree, don't store result of

$$\sigma_{building="Watson"}(department)$$

instead, pass tuples directly to the join.. Similarly, don't store result of join, pass **tuples** directly to projection.

Much cheaper than materialization: no need to store a temporary relation to disk.

Pipelining may not always be possible – e.g., sort, hash-join.

For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.

Pipelines can be executed in two ways: **demand driven** and **producer driven**

Pipelining (Cont.)

In **demand driven** (**lazy**, **pull**) evaluation

system repeatedly requests next tuple from top level operation

Each operation requests next tuple from children operations as required, in order to output its next tuple

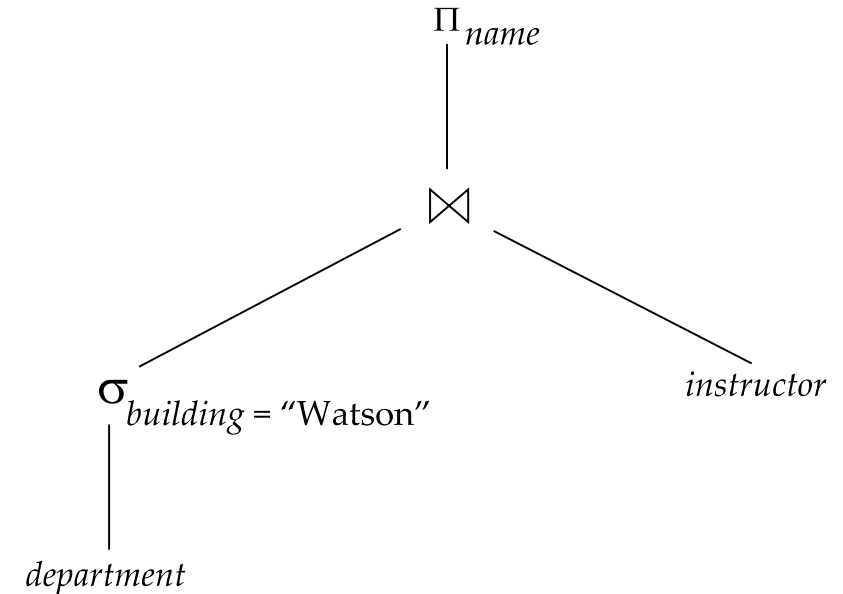
In between calls, operation has to maintain “**state**” so it knows what to return next

In **producer-driven** (**eager**, **push**) pipelining

Operators produce tuples eagerly and pass them up to their parents

- ▶ Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
- ▶ if buffer is full, child waits till there is space in the buffer, and then generates more tuples

System schedules operations that have space in output buffer and can process more input tuples



Pipelining (Cont.)

Implementation of **demand-driven pipelining**

Each operation is implemented as an **iterator** implementing the following operations

▶ **open()**

- E.g. file scan: initialize file scan
 - » state: pointer to beginning of file
- E.g. merge join: sort relations;
 - » state: pointers to beginning of sorted relations

▶ **next()**

- E.g. for file scan: Output next tuple, and advance and store file pointer
- E.g. for merge join: continue with merge from earlier state till next output tuple is found. Save pointers as iterator state.

▶ **close()**

Query Processing in Memory

Query compilation to machine code

Overheads of **interpretation**

- ▶ E.g., repeatedly finding attribute location within tuple, from metadata
- ▶ Overhead of expression evaluation

Compilation can avoid many such overheads and speed up query processing

Often via generation of Java byte code / **LLVM**, with **just-in-time (JIT)** compilation

Column-oriented storage

Allows **vector operations** (in conjunction with compilation)

Cache conscious algorithms

Cache Conscious Algorithms

Goal: minimize cache misses, make best use of data fetched into the cache as part of a cache line

For **sorting**:

- Use runs that are as large as L3 cache (a few megabytes) to avoid cache misses during sorting of a run

- Then merge runs as usual in merge-sort

For **hash-join**:

- First create partitions such that build+probe partitions fit in memory

- Then subpartition further s.t. build subpartition+index fits in L3 cache

 - Speeds up probe phase significantly by avoiding cache misses

Lay out attributes of tuples to maximize cache usage

- Attributes that are often accessed together should be stored adjacent to each other

Use multiple threads for parallel query processing

- Cache misses leads to stall of one thread, but others can proceed

End of Chapter 15